

AI wrote the SQL.

Your metrics are off by one day.

A practical guide to dates, timestamps,
time zones, and reporting periods.

Half-open ranges

DATE vs TIMESTAMP

Time zones

Week starts

Date spines

Window frames

Rolling vs calendar

Sagar Rathkanthiwar

Data & AI Professional

10 slides · save for later →

Use half-open ranges, not **BETWEEN**

BETWEEN is inclusive on both ends. On a timestamp column the upper end lands at midnight — so you silently lose the last day's activity.

```
-- RISKY: only catches 2026-08-31 00:00:00 exactly
WHERE order_ts BETWEEN '2026-08-01' AND '2026-08-31'
-- an order at 09:14 on Aug 31 is NOT counted

-- SAFE: half-open [start, next_start)
WHERE order_ts >= '2026-08-01'
      AND order_ts < '2026-09-01'
```

- The same shape scales up: a full year is `>= '2026-01-01' AND < '2027-01-01'`.
- It stays correct whether the column is a **DATE** or a **TIMESTAMP** — one habit, both types.
- Don't patch it with `<= '...23:59:59'`: that drops the final second, and misses rows entirely on microsecond precision.

Half-open ranges tile perfectly: August ends exactly where September begins. **No gaps, no double counting, no rounding to midnight.**

A date is a bucket.

A timestamp is an instant.

```
order_date  DATE           -- 2026-08-31          (no time)
order_ts    TIMESTAMP      -- 2026-08-31 23:47      (no zone)
created_at  TIMESTAMPTZ    -- 2026-08-31 23:47-07 (zone aware)

-- Compare a timestamp to a date and the date
-- is promoted to MIDNIGHT of that day:
WHERE order_ts <= DATE '2026-08-31'
-- really means order_ts <= 2026-08-31 00:00:00
```

- **DATE** — already a calendar bucket; safe to compare with `=`.
- **TIMESTAMP** — a wall-clock reading with no zone attached. Ambiguous on its own.
- **TIMESTAMPTZ / TIMESTAMP_TZ** — an absolute instant, normally stored as UTC.

Before you accept AI-generated SQL, ask one question: **"What is the exact type and stored time zone of every date column in this query?"**

One instant. Two different calendar days.

STORED IN UTC

**2026-04-01
06:30 UTC**

=

CUSTOMER SAW

**2026-03-31
23:30 PT**

Group that row by UTC day and a **March** order lands in your **April** report. Every late-evening order in the Americas is exposed to this.

```
-- WRONG: truncates in UTC
DATE_TRUNC('day', order_ts)

-- RIGHT: convert first, THEN truncate
DATE_TRUNC('day', order_ts AT TIME ZONE 'America/Los_Angeles') -- Postgres
DATE(order_ts, 'America/Los_Angeles')                         -- BigQuery
CONVERT_TIMEZONE('America/Los_Angeles', order_ts)            -- Snowflake
```

Convert **before** grouping, and use a named zone (`America/Los_Angeles`), never a fixed offset — named zones handle daylight saving; `-08:00` does not.

DATE_TRUNC('week')

is not a definition

The same call returns a **different first day** depending on the engine. AI will pick one; your finance team already picked another.

```
-- Postgres  DATE_TRUNC('week', d)           -> Monday, always
-- BigQuery  DATE_TRUNC(d, WEEK)             -> Sunday
--           DATE_TRUNC(d, WEEK(MONDAY))     -> Monday
-- Snowflake DATE_TRUNC('week', d)           -> WEEK_START param

-- Fiscal calendars can't be derived. Join them.
SELECT c.fiscal_year, c.fiscal_week, SUM(o.amount_usd)
FROM orders o
JOIN dim_calendar c ON o.order_date = c.date_day
GROUP BY 1, 2;
```

- Agree on the start day **before** writing the query, then encode it explicitly — never lean on the engine default.
- Two tools with different defaults produce two different week-over-week charts from identical data.

A 4-4-5 retail calendar, a 53-week fiscal year or a Sunday-start marketing week cannot be inferred from a date. Keep a calendar dimension and join to it.

Zero-activity days never show up

GROUP BY only returns days that exist in the data. A day with no orders produces no row — so the chart draws a straight line across the gap and averages divide by the wrong denominator.

```
-- Postgres: build a date spine, then LEFT JOIN to it
WITH days AS (
  SELECT GENERATE_SERIES(DATE '2026-08-01',
                        DATE '2026-08-31',
                        INTERVAL '1 day')::date AS day
)
SELECT d.day,
       COALESCE(SUM(o.amount_usd), 0) AS revenue
FROM days d
LEFT JOIN orders o ON o.order_date = d.day
GROUP BY 1 ORDER BY 1;
```

The spine goes on the **left** of the join and every measure gets `COALESCE(..., 0)`. BigQuery: `GENERATE_DATE_ARRAY`. Better still: a permanent `dim_date` table.

Name the window frame. Don't inherit it.

Leave the frame off and SQL defaults to `RANGE`, which treats **every row sharing a date as one peer group**. With duplicate dates, the output stops matching what people expect.

```
-- Ambiguous with duplicate dates: frame is implied
SUM(amount_usd) OVER (ORDER BY order_date)

-- SAFE: collapse to one row per day first...
WITH daily AS (
  SELECT order_date AS day, SUM(amount_usd) AS revenue
  FROM orders GROUP BY 1
)
SELECT day, revenue,
       SUM(revenue) OVER (
         ORDER BY day
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM daily;
```

One row per day, then an explicit `ROWS` frame. Feed it from the date spine on the previous slide so the total carries through quiet days.

“Last 30 days” is not “this month”

```
-- ROLLING: a moving 30-day window, ending today
WHERE order_ts >= CURRENT_DATE - INTERVAL '30 days'
      AND order_ts < CURRENT_DATE

-- CALENDAR: resets to zero on the 1st
WHERE order_ts >= DATE_TRUNC('month', CURRENT_DATE)
      AND order_ts < DATE_TRUNC('month', CURRENT_DATE)
                    + INTERVAL '1 month'
```

ROLLING

Stable trend line

Same window length every day, so movement is real movement.

CALENDAR

Partial and growing

On the 3rd you have 3 days of data.
Never compare it to a full prior month.

Both are correct — for different questions. **The bug is comparing one against the other** and calling it a decline.

6 questions for any AI-generated date query

- 1 Source time zone** — what zone is stored, and what zone should the report read in?
- 2 Boundaries** — inclusive or exclusive at each end? Is the range half-open?
- 3 Calendar definition** — when does a week, month or fiscal period start here?
- 4 Missing dates** — are zero-activity days present, or silently dropped?
- 5 Rolling vs. calendar** — moving window or period-to-date? Which did the stakeholder mean?
- 6 Which timestamp** — event time, ingestion time or last-updated time? Three different columns.

That last one is the quiet killer: filter on `updated_at` and yesterday's numbers **change tomorrow**. Late-arriving data needs event time.

THE TAKEAWAY

In SQL, time is never just a column

AI can generate the query.

You still have to define what the clock means.

- The syntax will be valid. The **business definition** is the part no model can read off your schema.
- Every bug in this deck produces a number that looks completely reasonable — which is exactly why they ship.
- So state the clock **in the prompt**: the zone, the boundaries, the calendar. Then verify it in the output.

**What's the most confusing time-related SQL bug you've hit?
Drop it in the comments — someone else is debugging it
right now.**

Sagar Rathkanthiwar · Data & AI Professional · follow for more field guides

Note on dialects: examples use PostgreSQL syntax unless labelled otherwise. BigQuery, Snowflake, Redshift, SQL Server and DuckDB differ in week-start defaults, time-zone conversion functions and date arithmetic — the underlying logic is the same; check your engine's docs for exact spelling.